

# GPQA 200-Question Verification Experiment

## Verification Generation Experiment

September 30, 2026

This report documents a 200-question GPQA Main feasibility experiment testing whether verifier signals can improve a generator's self-reported probability that its selected answer is correct. The experiment used a 120-question calibration partition and an 80-question held-out evaluation partition. Results are preliminary and do not establish robust calibration or incentive compatibility.

## Executive summary

The generator answered 69 of 80 evaluation questions correctly (86.3%; Wilson 95% interval 77.0%-92.1%). Four of 200 generator responses were invalid JSON; three occurred in evaluation. Verifier 1's update slightly worsened held-out Brier score. Verifier 2 and the sequential two-verifier update slightly improved Brier score, but paired bootstrap intervals included no change. The combined update slightly worsened log loss. The observed data do not establish a reliable improvement.

The held-out forecast comparison below uses the 77 evaluation questions with valid generator answers and confidence. Lower Brier score, log loss, and ECE are better; higher AUROC is better.

Forecast Brier Log loss ECE AUROC

Generator stated probability 0.0730 0.2575 0.0442 0.844

After Verifier 1 update 0.0737 0.2635 0.0476 0.824

After Verifier 2 update 0.0682 0.2542 0.0475 0.841

After both, sequentially 0.0679 0.2592 0.0490 0.819

## Dataset handling and partitioning

The released GPQA data are organized into variants such as Main, Diamond, and Extended rather than a conventional train/calibration/test split. The GPQA paper recommends Main for experiments; Diamond is a subset of Main, so it would not be an independent test if calibration also used Main. Sources: <https://arxiv.org/abs/2311.12022> and <https://huggingface.co/datasets/Idavidrein/gpqa>.

The experiment used the local gpqa\_main.csv file. Three rows had duplicate answer choices. They were excluded before sampling; the original CSV was not modified. The sample manifest records each excluded row's hashed item ID and the reason duplicate\_answer\_choices. The eligible pool therefore contained 445 questions.

We selected 200 questions from the eligible pool using deterministic subject stratification and seed 20260928. Answer choices were also shuffled deterministically, with the correct option position updated accordingly. The 200 sampled questions were split into 120 calibration items and 80 evaluation items. Their hashed IDs are disjoint. Calibration labels were used to fit verifier-signal likelihoods; the held-out evaluation labels were used only to score forecasts and policies.

This is a random split within GPQA Main, not an official GPQA test set or an external generalization test. Model exposure to public GPQA questions is also possible.

## Models and generation settings

All inference used Google Vertex AI managed model endpoints.

- \* Generator: google/gemini-3.8-flash.
- \* Verifier 1: meta/llama-3.3-70b-instruct-maas.
- \* Verifier 2: google/gemini-3.7-flash.

The configured generation settings were temperature 0, top-p 1, maximum output length 1,024 tokens, and low reasoning effort for Gemini models. These are managed model names rather than immutable weight revisions, so exact reproduction may be affected if the provider changes an alias.

## How probabilities were obtained

The generator saw the subject, question, and four answer options. It was asked to choose one option and report JSON with its probability that its selected answer was correct. This is a self-reported textual probability, not a probability computed from token likelihoods.

Each verifier saw the same question and options plus the fixed answer selected by the generator. It was asked to assess that answer, not replace it, and return a probability that the fixed answer was correct. The verifier did not receive the generator's reasoning; the prompt included the candidate answer and its option text.

The response parser accepted a valid answer letter and a numeric probability in [0,1]. Invalid JSON, answer letters, or probabilities were recorded as failures. A verifier was called only when the generator's answer letter could be parsed. Invalid generator answers count as incorrect in answer accuracy; rows without valid confidence are omitted from probability forecast metrics.

## How the Brier scores were calculated

For every held-out item with a valid generator answer and confidence, let  $p_i$  be the probability being evaluated and let  $y_i$  equal 1 when that selected answer matches the GPQA answer key, or 0 otherwise. The Brier score is the mean squared probability error:  $\text{Brier} = (1/N) \sum_i (p_i - y_i)^2$ . A forecast of 0.95 scores  $(0.95-1)^2 = 0.0025$  when correct, and  $(0.95-0)^2 = 0.9025$  when incorrect. Lower scores are better.

The generator Brier score of 0.0730 is computed directly from its reported  $p_{\text{correct}}$  values, before verification, over 77 held-out questions. Each raw verifier Brier score instead uses that verifier's reported  $p_{\text{correct}}$  for the fixed candidate. Each posterior Brier score uses the confidence after applying the calibration-set update. All these scores use the same answer-key outcome definition. Invalid generator JSON has no valid probability and is not included in Brier calculations, although it counts as an incorrect/invalid answer for accuracy.

## Calibration and Bayesian combination

For each verifier separately, calibration items with a valid generator answer and confidence were labeled by whether the selected answer matched the shuffled correct-answer position. The verifier's numeric score was placed into one of three equal-width bins over [0,1]. From calibration data, the pipeline estimated  $P(\text{score bin} \mid \text{correct})$  and  $P(\text{score bin} \mid \text{incorrect})$ , adding one Laplace pseudo-count to each bin in each class.

For generator confidence  $p$  and verifier score  $s$  in bin  $b$ , the posterior was computed as:

$$P(\text{correct} \mid b) = p P(b \mid \text{correct}) / [p P(b \mid \text{correct}) + (1-p) P(b \mid \text{incorrect})].$$

The generator's stated confidence is the prior. Calibration estimates the verifier's likelihood signal; it does not directly replace the generator's answer or fit a more flexible neural model. Exact generator probabilities of zero or one cannot be changed by a Bayesian update.

For the two-verifier result, the pipeline updated the generator prior first with Verifier 1 and then with Verifier 2, using each verifier's separately fitted likelihood table. This sequential product assumes conditional independence of the verifier signals given correctness. That assumption is not established. In the full sample, generator and Verifier 2 residual scores were strongly associated (correlation about 0.78), so the combined result should be treated as exploratory. The runner collected both verifiers for all parseable candidate answers; the sequential-stopping policy is a counterfactual analysis of those signals, not a reduction in actual collection calls.

Numerical example from the fitted calibration tables: for a generator prior  $p=0.95$  and Verifier 1 score 0, the score falls in the first bin. Calibration estimated  $P(\text{bin} \mid \text{correct})=0.327$  and  $P(\text{bin} \mid \text{incorrect})=0.267$ , giving an updated probability of about 0.959. The Verifier 1 signal changes this prior very little. For the same  $p=0.95$  and Verifier 2 score 0, the corresponding estimates were 0.046 and 0.267; the updated probability is about 0.766. These are fitted examples, not a claim that either score or update is reliable for an individual question.

## Relation to the Green-Laffont algorithm

The project specification cites Sections 2.4-2.5 of [green-laffont.pdf](#) for noise-corrected confidence elicitation and nested sequential verification. That PDF was not present in the repository, so this is a correspondence to the algorithm as summarized in the project specification, not a line-by-line audit of the paper.

The implemented pilot corresponds to these operational steps: start from a probability that the fixed candidate answer is correct; model each verifier output as a noisy signal whose distribution depends on whether that candidate is correct; update the belief by Bayes' rule; and compare stopping against paying for another verifier. The routing code computes a dynamic-programming value recursion over answer, abstain, or query actions, using the configured reward  $R$ , incorrect-answer loss  $L$ , and verifier costs. Its release threshold after observations is  $L/(R+L)$ .

In simplified notation, the stopping value is  $S(p)=\max(0, pR-(1-p)L)$ , where 0 is abstention. At verifier stage  $k$ , the continuation value is  $C_k(p)=-c_k+\sum_z P(z|p)V_{k+1}(\text{Bayes}(p,z))$ ; the policy takes the larger of  $S(p)$  and  $C_k(p)$ . The implementation discretizes posterior beliefs for memoized recursion. This value calculation is evaluated after responses have been collected, so it does not currently control live API calls.

The experiment does not implement the paper's complete mechanism or test its incentive-compatibility claim. It does not train strategic agents to report confidence under payments, does not implement noise-corrected elicitation/payment rules, and does not test a terminal audit. Instead, confidence and verifier scores are ordinary prompted text from frozen models, while answer-key correctness is used offline for calibration and evaluation. Both verifiers were collected for every parseable answer; dynamic-programming stopping and query savings are counterfactual policy analyses. The likelihood update for two verifier signals also uses a naive conditional-independence approximation, which the measured dependence cautions against.

## Results

### Answer and parse outcomes

Across all 200 sampled items, 176 answers were correct (88.0%) and 24 were incorrect or invalid. The generator response parser accepted 196 responses and rejected four as invalid JSON (2.0%). The calibration partition had 107 correct answers out of 120; one generator response was invalid JSON. The evaluation partition had 69 correct answers out of 80; three generator responses were invalid JSON. Evaluation answer accuracy had a Wilson 95% interval of 77.0%-92.1%.

## Calibration data

Verifier 1 had 119 usable calibration records: 107 correct and 12 incorrect. Verifier 2 had 118: 106 correct and 12 incorrect; one additional verifier response was invalid JSON. Twelve incorrect examples per verifier provide limited information for estimating error-conditioned score likelihoods.

## Held-out forecast metrics

Among the 77 evaluation items with valid generator confidence, the generator's Brier score was 0.0730 (bootstrap 95% interval 0.0294-0.1236), log loss 0.2575 (0.1149-0.4180), ECE 0.0442, and AUROC 0.844 (0.675-0.978).

After Verifier 1, Brier score was 0.0737 and log loss 0.2635. The paired change versus generator confidence was +0.00068 for Brier (95% interval -0.00047 to +0.00247) and +0.00602 for log loss (-0.00247 to +0.01530). Neither change is distinguishable from zero in this sample.

After Verifier 2, Brier score was 0.0682 and log loss 0.2542. The paired changes were -0.00475 for Brier (95% interval -0.02293 to +0.00964) and -0.00329 for log loss (-0.04832 to +0.03404). These intervals also include zero.

After sequentially applying both verifier signals, Brier score was 0.0679 and log loss 0.2592. Brier was modestly lower than the generator's 0.0730, while log loss was modestly higher than 0.2575. This comparison depends on the unverified conditional-independence assumption, and there is no paired confidence interval for this combined change in the report.

Verifier 1 raw evaluation scores were extreme: 28 of 77 were zero and 49 were one. Its raw Brier score was 0.364 and raw log loss 13.107. Verifier 2 raw Brier score was 0.091, log loss 3.277, ECE 0.090, and AUROC 0.668. These are diagnostics of the raw self-reports, before the calibration likelihood update.

## Usage, artifacts, and validation

The run logged 200 generator responses, 196 Verifier 1 responses, and 196 Verifier 2 responses (592 responses total). The four generator parse failures received no verifier calls. Logged token totals were approximately 53,535 generator prompt tokens, 4,436 completion tokens, and 110,104 reasoning tokens; 61,758 prompt and 1,960 completion tokens for Verifier 1; and 54,563 prompt, 2,953 completion, and 81,198 reasoning tokens for Verifier 2. Billing cost is not estimated here.

The run ID is a5d92086584910dbe156297b5c1083bb8bf47765670b46a0dfc1b7ddc49cbce7. Artifacts are under results/gpqa/a5d92086584910dbe156297b5c1083bb8bf47765670b46a0dfc1b7ddc49cbce7, including pilot\_records.jsonl, pilot\_metrics.json, run\_manifest.json, and per-model call logs. The pinned sample manifest is data/gpqa/sample\_200.json. The synthetic software test suite passed 107 tests with 15 skipped; those tests validate pipeline behavior and do not demonstrate real verifier calibration.

## Limitations and next step

The evaluation contains only 80 questions and 11 incorrect or invalid answers. The calibration set contains only 12 valid incorrect outcomes per verifier. These sizes leave substantial uncertainty, especially for verifier likelihood estimates and tail probabilities. The generator's probabilities are self-reported and clustered toward high confidence; verifier 1's raw scores are nearly binary. The combined update relies on a conditional-independence assumption contradicted by observed score dependence. The public benchmark may also have appeared in model training data. Results support feasibility diagnostics only; they do not show reliable calibration, incentive compatibility, or transfer to new tasks.

A stronger follow-up should pre-register the calibration procedure and evaluation measures, then evaluate once on a larger untouched question pool or another non-overlapping dataset. It should preserve the held-out split from all prompt, threshold, and model selection, assess verifier dependence explicitly, and use immutable model versions where available.